

Hệ thống biên tập / 2026

MẪU DUYỆT — không phải PDF phát hành. Nội dung bài học lấy từ Markdown hiện hành; PDF khóa chỉ là đối chứng hình thức.

Bốn hướng bìa, một hệ thống trang ruột.

Giữ body serif 14 pt, code 11.5 pt, nền trắng và lề đối xứng. Đề xuất làm rõ phân cấp, giữ lưới bảng, tăng chữ Atlas và đưa sơ đồ phù hợp sang vector. Không dùng 3D để trang trí một cơ chế kỹ thuật.

Bản mẫu chưa đạt cổng copy/paste code: TAB và whitespace không có bảo đảm chỉ nhờ PDF text hoặc ActualText. Hai thư viện extract không thay thế hai PDF reader.

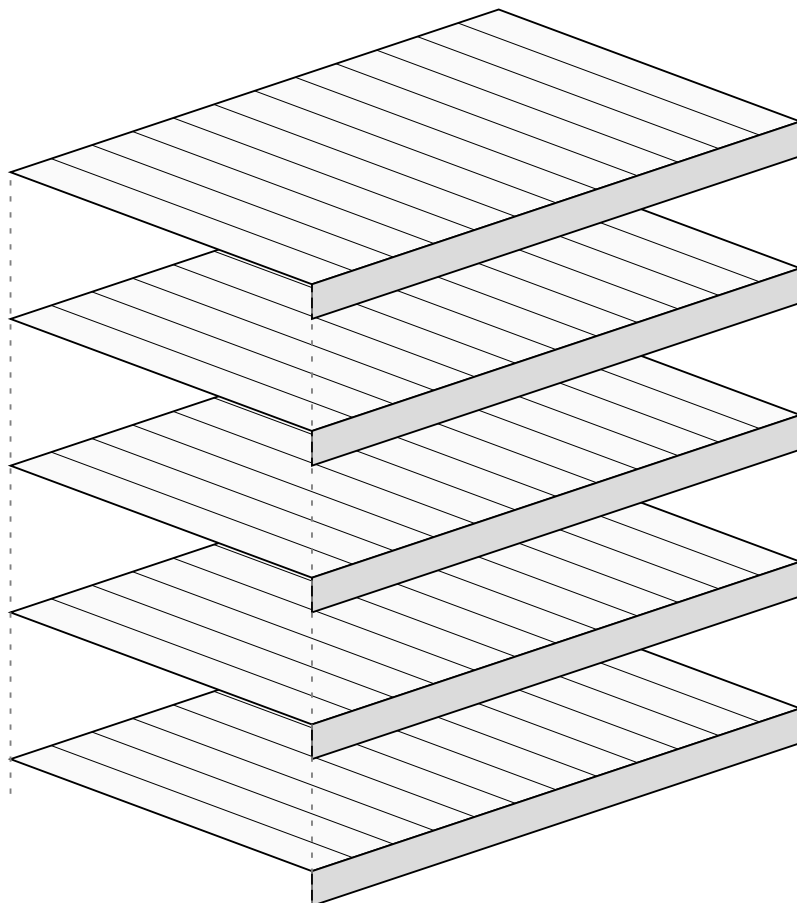
Điều hướng

Bìa A — exploded axonometric	Xem mẫu
Bìa B — architectural ink study	Xem mẫu
Bìa C — scientific contour	Xem mẫu
Bìa D — experimental typography	Xem mẫu
Quy chuẩn, bìa sau và gáy	Xem mẫu
Mở chương và trang phối hợp	Xem mẫu
Văn xuôi / vector / caption	Xem mẫu
Câu hỏi	Xem mẫu
Đáp án	Xem mẫu
Library Atlas	Xem mẫu
Error Atlas	Xem mẫu
Code-copy laboratory	Xem mẫu

GOLANG

BÌA A / Exploded axonometric / PROTOTYPE

Giáo trình cập nhật liên tục về Kỹ nghệ phần mềm và DevOps/SRE



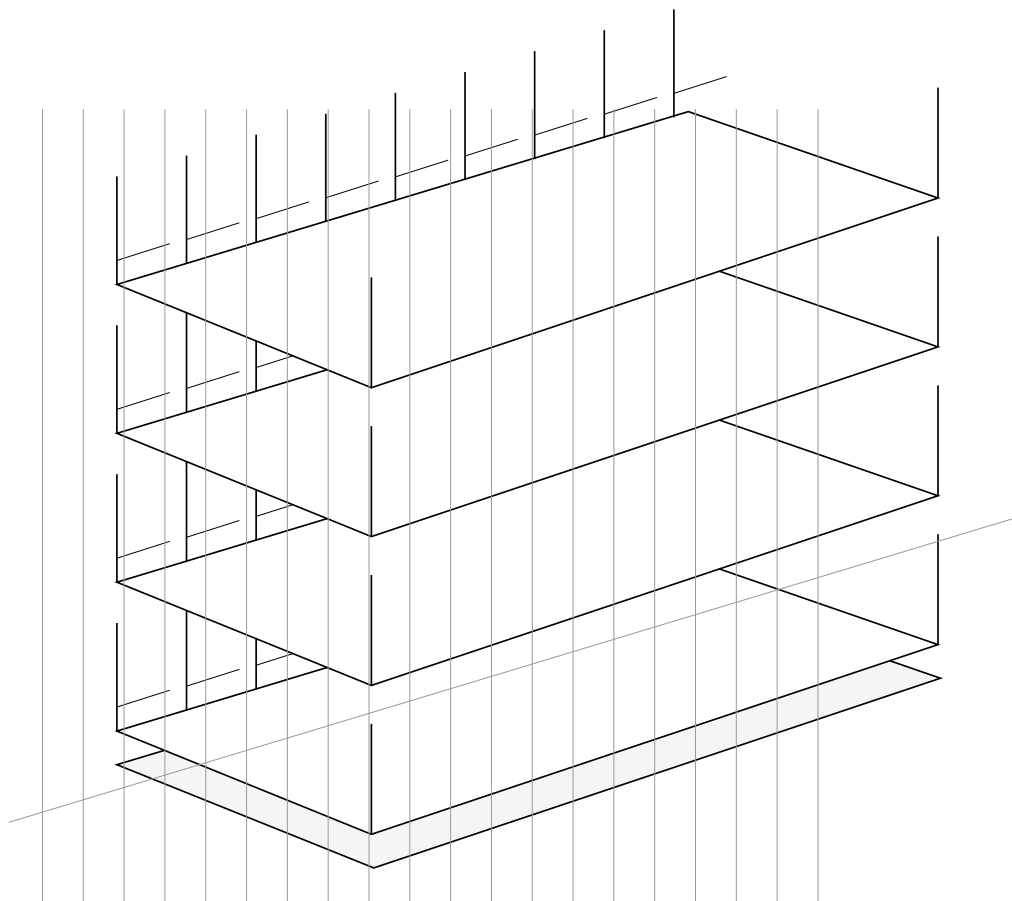
Đoàn Ngọc Hoàng Minh

Các lớp tách rời; dễ đọc bằng nét và khoảng trắng. Đây là artwork trừu tượng, không phải sơ đồ runtime.

GOLANG

BÌA B / Architectural ink study / PROTOTYPE

Giáo trình cập nhật liên tục về Kỹ nghệ phần mềm và DevOps/SRE



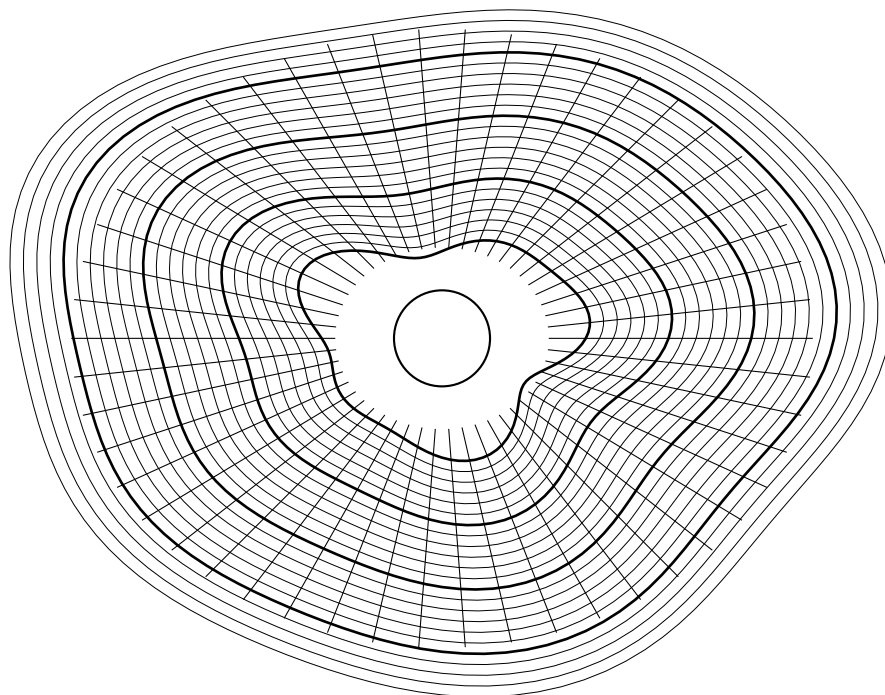
Đoàn Ngọc Hoàng Minh

Cấu trúc phối cảnh với nét cắt và chi tiết. Mật độ nét cao hơn; cần proof giấy trước khi chốt.

GOLANG

BÌA C / Scientific contour / PROTOTYPE

Giáo trình cập nhật liên tục về Kỹ nghệ phần mềm và DevOps/SRE



Đoàn Ngọc Hoàng Minh

Vòng đồng mức và lớp tiết diện trừu tượng; không đại diện một cấu trúc bộ nhớ Go.

GOLANG

BÌA D / Experimental typography / PROTOTYPE

Giáo trình cập nhật liên tục về Kỹ nghệ phần mềm và DevOps/SRE

GO
LANG

A series of thin, vertical black lines of varying heights, creating a rhythmic, barcode-like pattern that spans the width of the page below the word 'LANG'.

NGÔN NGỮ / CƠ CHẾ / BẢNG CHỨNG

Đoàn Ngọc Hoàng Minh

Chữ là hình chính; ít mực, ít rủi ro mất chi tiết. Chỉ bìa được phép chơi với tỷ lệ chữ.

Một hệ thống, không nhiều khuôn

Quy chuẩn trang ruột dùng chung; artwork không quyết định phương pháp dạy từng chương.

Mở phần / Library Atlas

Chương 2 — Khi một bản sao vẫn chia sẻ dữ liệu

Một baseline để dự đoán

Chẩn đoán bằng quan sát

Đáp án — chỉ đọc sau khi đã tự làm

Mẫu H3 trên chỉ là nhãn thử kiểu chữ, không phải nội dung mới của sách.

Object	Quy chuẩn chính thức đề xuất
Prose	Source Serif 4 / 14 / 21.5 pt / đen
H1 / H2 / H3 / H4	Sans semibold / 24–17–14–12.5 pt
Code / terminal	JetBrains Mono / 11.5 / 15.5 pt; không gutter
Table / caption	11 / 15.5 pt; 10.5 / 14 pt; giữ grid
Atlas	10.5 / 14 pt; hai cột, gutter 8 mm
Geometry	A4; inner 24, outer 18, top 22, bottom 20 mm

Bìa sau / gáy / mở phần

Bìa sau và gáy hiện chưa có trong PDF khóa. Không tự đặt ISBN, blurb hoặc chiều dày gáy.

GOLANG

Đoàn Ngọc Hoàng Minh

PLACEHOLDER — vùng bìa sau để duyệt cấu trúc. Không có nội dung quảng bá mới.

PLACEHOLDER — gáy phụ thuộc định lượng/độ dày giấy, số tờ và kiểu đóng. UNKNOWN; không đưa kích thước minh họa thành thông số in.

BACK MATTER — ATLAS MÃ NGUỒN 50 THƯ VIỆN GO DEVOPS & CLOUD

Phụ bản này đứng sau Chương 29 và ngay trước Atlas Lỗi Go. Các nhận xét về implementation chỉ áp dụng cho version và commit ghi ở mỗi mục, không phải cam kết API cho mọi version. Khóa source chứng minh identity của checkout, không tự chứng minh mọi giải thích cơ chế; khi dùng một chi tiết để quyết định production, đọc code và test tại commit đó.

Mở chương: từ kết quả đến câu hỏi

Trích liên tục đầu Chương 2; không dùng một template dạy học mới.

Chương 2 — Khi một bản sao vẫn chia sẻ dữ liệu

Chạy đoạn này trước khi cố giải thích nó. Nếu bạn đoán a và b khác nhau sau dòng `b[0] = 99`, đó là một dự đoán rất hợp lý - và cũng là cánh cửa vào phần này.

```
a := []int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a)
fmt.Println(b)
```

Kết quả từ lab part2-values là:

```
[99 20 30]
[99 20 30]
```

Đừng ghi nhớ riêng một câu “slice nguy hiểm”. Thay vào đó, ta sẽ lần ngược từ kết quả: Go đã copy cái gì khi chạy `b := a`, và dữ liệu nào không được copy?

Văn xuôi / code / bảng

Hai đoạn trích giữ thứ tự nguồn; dấu ngắt dưới đây báo phần giữa đã lược trong catalog, không lược trong bản thảo.

Với một int, assignment tạo ra một giá trị độc lập. Cả a và b đều có giá trị riêng của chính chúng.

```
a := 10
b := a
b = 99

fmt.Println(a, b) // 10 99
```

Go đã copy value của a vào b. Sau đó b nhận value mới; a không có lý do gì để đổi. Đây là quy tắc nền: assignment copy value. Điều còn thiếu là một value có thể chứa gì.

[NGẮT TRÍCH — xem phần liên tục trong bản thảo]

Copy value và chia sẻ storage là hai tầng khác nhau của cùng một phép gán.

Câu nói	Đúng ở đâu?
Assignment copy value.	b nhận một slice descriptor riêng: đổi len của b bằng reslice không tự đổi len của a.
Hai slice có thể cùng đổi dữ liệu.	Slice descriptor cùng tham chiếu một backing array; mutation một element hiện ra qua mọi slice cùng nhìn element đó.

Từ aliasing dùng cho tình huống nhiều đường tham chiếu đến cùng dữ liệu. Nó không phải lỗi tự thân. Aliasing giúp cắt một cửa sổ trên buffer mà không copy cả buffer. Nó thành bug khi một người đọc tưởng mình có dữ liệu riêng nhưng thực tế lại đang sửa vùng dùng chung.

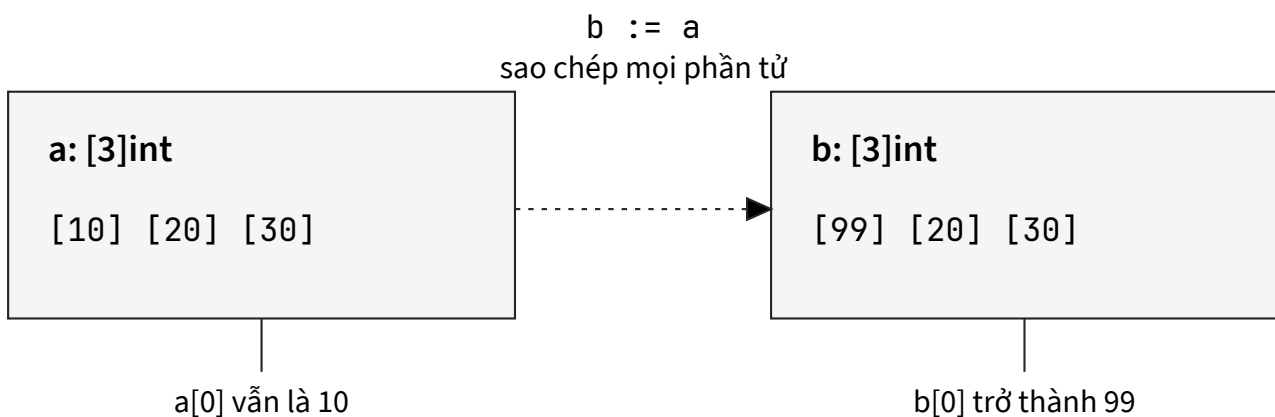
Văn xuôi / vector / caption

Hai nút, một cạnh copy nét đứt có hướng và hai ghi chú: giữ nguyên toàn bộ nhãn và quan hệ từ array-copy.puml.

Array giúp ta kiểm tra quy tắc ấy với nhiều phần tử. Độ dài là một phần của type, nên `[3]int` và `[4]int` là hai array type khác nhau. Khi array được gán, toàn bộ array value được copy.

```
a := [3]int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a) // [10 20 30]
fmt.Println(b) // [99 20 30]
```

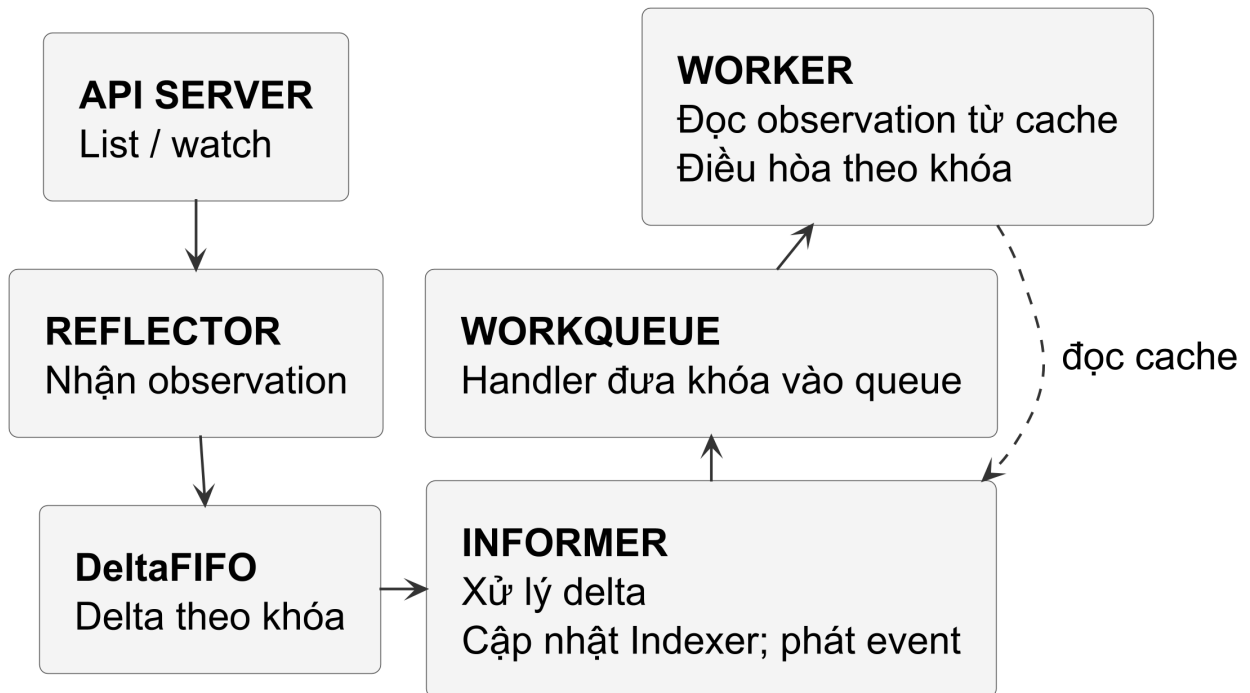


Assignment của array copy các phần tử. Việc thay `b[0]` chỉ chạm vào array `b`.

Đến đây, trực giác “copy nghĩa là độc lập” vẫn đúng. Sự thay đổi xuất hiện chỉ khi ta bỏ một ký tự trong type: `[3]int` trở thành `[]int`.

Sơ đồ phức tạp: giữ nguồn trước

KEEP có chủ đích: không dùng một bộ tự chuyển tất cả hình sang 3D hoặc vector mà chưa đối chiếu nhãn và cạnh.



Informer cập nhật cache rồi phát event tới handler; handler của ứng dụng đưa khóa vào workqueue. Worker đọc observation hiện có khi điều hòa, không coi payload event là trạng thái cuối cùng.

PNG này chỉ minh họa phương án KEEP và kích thước đọc. DPI hữu hiệu được đo trong qa_results.json. Vector hóa diagram này là hạng mục tích hợp sau phê duyệt, không tự nhận đã hoàn tất.

Câu hỏi trước đáp án

Giữ điểm dừng sự phạm; đáp án chỉ xuất hiện ở trang kế tiếp.

Dừng để dự đoán. Đọc đoạn mã sau và dự đoán chính xác hai dòng output: biến x trong block con nhận giá trị nào? Sau khi thoát khỏi block con, x có giá trị là 1, 2 hay 4?

```
x := 1
x, y := 2, 3
{
    x := 4
    fmt.Println(x, y)
}
fmt.Println(x, y)
```

Đọc lại sau khi tự dự đoán

Đáp án giữ nguyên văn; H4 là mẫu xử lý riêng, không để literal ##### rò ra trang.

Đáp án — chỉ đọc sau khi đã tự làm

Kết quả in ra lần lượt là 4 3 rồi 2 3. Tại câu lệnh `x, y := 2, 3`, do xuất hiện trong cùng block và có thêm biến mới `y`, Go áp dụng quy tắc `redeclaration`: biến `x` cũ nhận giá trị mới là 2 mà không tạo ra bản sao thứ hai. Khi bước vào block `con`, khai báo ngắn `x := 4` tạo ra một biến hoàn toàn mới nằm trong phạm vi từ vựng hẹp hơn, che khuất biến ngoài và khiến lệnh in đầu tiên xuất ra 4 3. Sau khi thoát khỏi block `con`, biến nội bộ này hết phạm vi từ vựng; lệnh in cuối cùng đọc lại biến `x` của block cha (đang giữ giá trị 2) và biến `y` (3), cho kết quả 2 3.

Phản ví dụ biên giới làm rõ bản chất: nếu bỏ `y` để viết tiếp `x := 2` trong cùng block, compiler lập tức báo lỗi `no new variables on left side of :=`. Ngược lại, phép gán `x = 2` thì hợp lệ vì nó chỉ cập nhật giá trị của biến sẵn có. Phạm vi từ vựng, không phải hình thức của cú pháp `:=`, quyết định biến nào được dùng lại.

Bảng: lưới phục vụ phân biệt dữ liệu

Bảng zero value nguyên văn Chương 1; không bỏ đường kẻ để chạy theo hình thức.

Kiểu dữ liệu	Giá trị Zero Value theo Đặc tả Ngôn ngữ	Ghi chú ngữ nghĩa
Số nguyên (int, int64, byte...)	0	Giá trị số không.
Số thực (float32, float64)	0.0	Số không dấu phẩy động.
Logic (bool)	false	Trạng thái sai mặc định.
Chuỗi ký tự (string)	""	Chuỗi rỗng với độ dài bằng 0.
Con trỏ, slice, map, channel, func, interface	nil	Zero value có ý nghĩa và giới hạn thao tác riêng theo từng kiểu; Chương 2-3 lần rõ các trường hợp dữ liệu.

Library Atlas: cơ chế có điểm neo

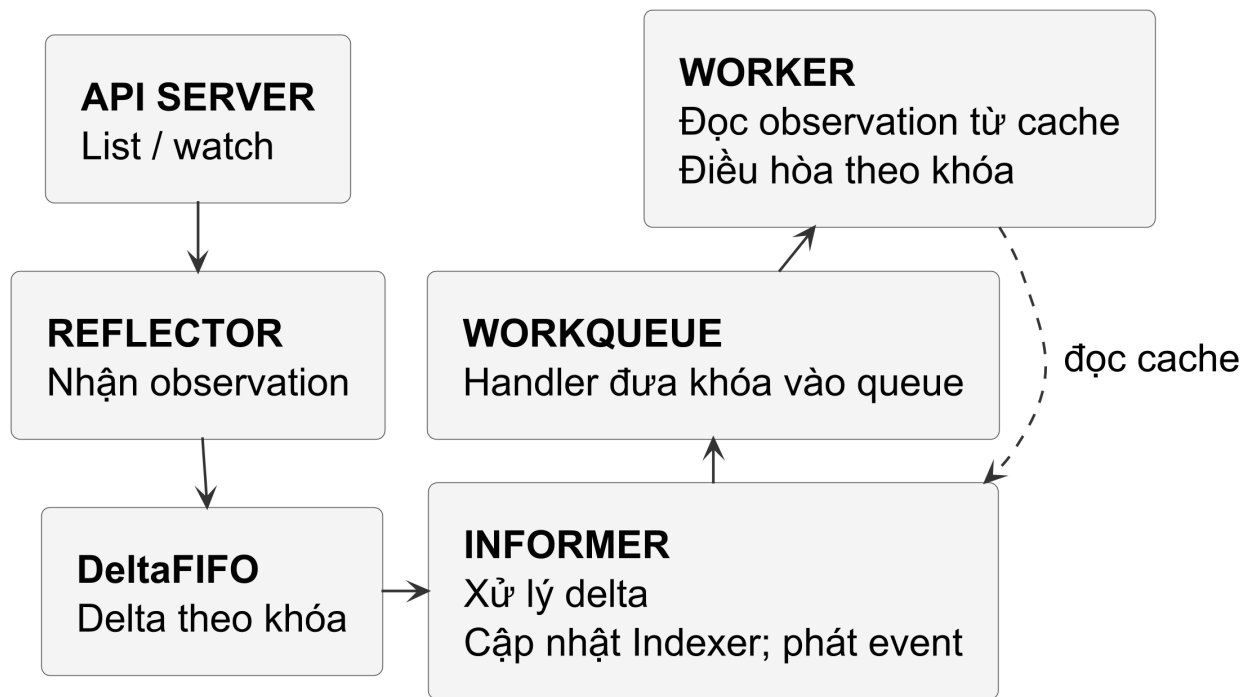
Trích liên tục mục 01 từ source hiện hành. Không đổi version, số liệu hoặc nhận xét implementation.

01. `k8s.io/client-go` (v0.37.0 — 28076445)

Nếu controller lập List toàn bộ Pod mỗi giây, nó tạo công việc tuần tự hóa và truyền snapshot lặp lại. Mức tải phụ thuộc số object, kích thước và API server; không suy ra CPU 100% hay cluster sập chỉ từ 500 Pod. Informer hỗ trợ duy trì observation local từ list/watch để nhiều consumer không phải tự polling toàn bộ tài nguyên theo cùng nhịp.

Reflector duy trì observation bằng list/watch và xử lý thay đổi resourceVersion cùng lỗi transport. Đường khởi tạo cụ thể có thể dùng List hoặc watch-list tùy version, capability và cấu hình. Watch là stream API event, không phải trực tiếp theo dõi mọi thay đổi etcd; HTTP/2 cũng không dùng HTTP/1.1 chunked transfer encoding. Khi lịch sử không còn đủ hoặc stream lỗi, Reflector xử lý recovery theo source đã pin; không phải mỗi lần đứt stream đều bắt buộc List lại.

Dữ liệu nhận về không được chuyển ngay cho người dùng mà đẩy vào một hàng đợi gom cụm đột biến mang tên `DeltaFIFO` (`tools/cache/delta_fifo.go`). Khi một Pod bị sửa đổi dồn dập 10 lần trong nửa giây, `DeltaFIFO` lưu trữ chuỗi delta theo khóa định danh `namespace/name`. Từ đây, vòng lặp xử lý nội bộ của Informer (`controller.processLoop`) rút delta ra, cập nhật đối tượng vào một bộ nhớ đệm luồng an toàn cục bộ gọi là `Indexer` (`tools/cache/index.go`), được bảo vệ bởi `sync.RWMutex`.



Informer cập nhật cache rồi phát event tới handler; handler của ứng dụng đưa khóa vào workqueue. Worker đọc observation hiện có khi điều hòa, không coi payload event là trạng thái cuối cùng.

Điểm neo kỹ thuật đáng giá nhất để học trong `client-go` nằm ở `util/workqueue/queue.go` (type `Typed[T]`). `TypedInterface[T]` là giao diện public; `TypedQueueConfig[T]` là cấu hình. Triển khai `Typed[T]` phối hợp hàng đợi có thể thay thế và hai tập khóa:

```

type Typed[T comparable] struct {
    queue      Queue[T]
    dirty      sets.Set[T]
    processing sets.Set[T]
    cond       *sync.Cond
    // Các trường lifecycle và metrics khác được lược bỏ.
}

```

Hãy hình dung: Worker đang bận xử lý key "default/my-pod" trong tập `processing`. Cùng lúc đó, 5 sự kiện cập nhật liên tiếp của Pod này được Informer gửi tới. `Add(item)` chỉ đánh dấu key trong `dirty` khi nó đang được xử lý, không đẩy thêm vào hàng đợi. Khi worker gọi `Done(item)`, nếu key vẫn `dirty`, nó được đẩy lại đúng một lần. Cơ chế khử trùng lặp này giới hạn việc xếp trùng một key; thứ tự và cấu trúc lưu trữ phụ thuộc triển khai `Queue[T]`. Worker thường lấy key rồi đọc snapshot hiện tại từ Indexer —

snapshot đó có thể stale so với API server. Reconciliation lặp lại mới giúp hội tụ về trạng thái mong muốn.

Khi worker không chạy, kiểm tra lifecycle, error của Reflector và cache sync. Thiếu quyền list có thể ngăn initial population; thiếu quyền watch ảnh hưởng nhận cập nhật. Không suy ra HasSynced luôn false chỉ từ một lỗi watch, vì trạng thái sync còn phụ thuộc đường khởi tạo. Phải có cancellation/deadline và quan sát error thay vì chờ vô hạn.

Error Atlas / bản tra cứu

Mẫu hai cột: giữ cả entry. Body 10.5 pt; không giới hạn số trang. Hai trích đoạn A01–A14 và J06–J11, không phải phụ lục đầy đủ.

A01 undefined: x

Không tìm thấy identifier trong scope hiện tại hoặc identifier chưa được export từ package ngoài.

→ Kiểm tra lỗi chính tả tên biến/hàm, scope khai báo, hoặc viết hoa chữ cái đầu nếu gọi từ package khác. [Ch1,3,5]

A02 no new variables on left side of :=

Tất cả các biến ở vế trái toán tử := đều đã được khai báo trước đó trong cùng một scope.

→ Đổi toán tử sang phép gán thường =, hoặc bổ sung ít nhất một biến mới vào vế trái. [Ch1,2]

A03 "fmt" imported and not used

Diagnostic này được compiler Go 1.27.1 phát cho import fmt không được dùng; tên package khác làm chuỗi thay đổi. Dot import cũng cần dùng identifier của package, còn blank import khai báo chủ đích chỉ chạy initialization.

→ Xóa dòng import thừa, dùng blank identifier _ "pkg" nếu cần chạy hàm init(), hoặc chạy goimports. [Ch1,5]

A04 declared and not used: x

Biến cục bộ được khai báo nhưng không có bất kỳ dòng code nào tiếp theo đọc hay sử dụng giá trị của nó.

→ Sử dụng biến vào logic, gán vào blank identifier _ = x nếu đang debug dở, hoặc loại bỏ khai báo thừa. [Ch1,2]

A05 cannot use x (variable of type T1) as T2 value in argument

Sai khác kiểu dữ liệu trong phép gán hoặc truyền đối số; Go không hỗ trợ ép kiểu ngầm định (implicit coercion).

→ Thực hiện ép kiểu tường minh T2(x) nếu hai kiểu tương thích, hoặc sửa signature của hàm nhận. [Ch2,3,14]

A06 assignment mismatch

- 1 variable but f returns 2 values
- 2 variables but f returns 1 value

Số lượng biến nhận ở vế trái không khớp chính xác với số lượng giá trị trả về của hàm ở vế phải.

→ Kiểm tra signature của hàm gọi và hứng đủ tất cả các giá trị trả về (dùng _ để bỏ qua nếu cần). [Ch1,4]

A07 missing return

Hàm có khai báo kiểu dữ liệu trả về nhưng tồn tại nhánh rẽ control-flow đi đến cuối block mà không có lệnh return.

→ Rà soát các nhánh if/else, switch/case để bảo đảm mọi luồng thực thi đều trả về giá trị hoặc panic. [Ch1,4]

A08 invalid operation: operator not defined on struct

Sử dụng toán tử so sánh (==, !=) trên struct chứa ít nhất một field không thể so sánh (như slice, map, func).

→ So sánh từng field có thể so sánh, hoặc tự viết hàm helper so sánh nghiệp vụ chuyên biệt. [Ch2,3]

A09 cannot assign to struct field in map

Không thể gán trực tiếp m["k"].Field vì map index expression không addressable theo Go spec; đừng suy ra quy tắc này từ một địa chỉ vật lý của implementation.

→ Lấy struct ra biến tạm, cập nhật field rồi gán ngược lại vào map, hoặc dùng map chứa pointer map[K]*V. [Ch2,3]

A10 cannot take the address of x

Toán tử lấy địa chỉ & được áp dụng lên unaddressable value (giá trị trả về của hàm, hằng số, map index expression).

→ Gán giá trị đó vào một biến tạm cục bộ trước khi thực hiện lấy địa chỉ &temp. [Ch2,14]

A11 constant overflows int

Giá trị hằng số vượt quá giới hạn biểu diễn bit của kiểu số nguyên đích trong kiến trúc hiện tại.

→ Chỉ định kiểu số nguyên có độ rộng bit lớn hơn (int64, uint64), hoặc sử dụng package math/big. [Ch1,2]

A12 syntax error: unexpected newline, expecting comma or }

Thiếu dấu phẩy , ở phần tử cuối cùng của composite literal hoặc danh sách tham số nhiều dòng (multi-line).

→ Bổ sung dấu phẩy , vào cuối dòng ngay trước dấu đóng ngoặc nhọn } hoặc ngoặc đơn). [Ch1,3]

A13 invalid type assertion: x.(T)

Thực hiện cú pháp type assertion trên một biến không phải kiểu interface (non-interface type).

→ Chỉ áp dụng type assertion trên biến kiểu interface; kiểm tra lại kiểu dữ liệu thực tế của biến x. [Ch3,14,19]

A14 x does not implement I (missing method M)

- method M has pointer receiver

Type cụ thể không thỏa mãn interface vì thiếu method hoặc sai khác receiver (pointer receiver vs value receiver).

→ Kiểm tra receiver của method M: nếu là pointer receiver *T, đối tượng truyền vào phải là địa chỉ &val. [Ch3,5,19]

[NGẮT TRÍCH — B đến J05 vẫn còn nguyên trong source]

J06 Liveness probe failed

Liveness probe thất bại; đạt failure threshold dẫn tới xử lý restart container theo lifecycle và restart policy. Một lần probe lỗi chưa đủ suy ra container bị restart ngay.

! Tránh dùng lỗi database xa làm liveness failure nếu restart không chữa được nó; policy sai có thể gây cascading restart.

→ Chọn probe mà restart có khả năng chữa failure. Thường tránh đưa dependency xa vào liveness nếu nó gây restart dây chuyền; đây là khuyến nghị thiết kế, không phải giới hạn API chỉ cho kiểm tra nội tại. [Ch12,17,21]

J07 Admission Gate Rejected

- unsigned artifact or digest mismatch

Trạng thái từ chối (Admission / Policy Status) từ Kubernetes Validating Admission Webhook hoặc Promotion Gate của CI/CD khi artifact container image không có chữ ký cosign hợp lệ hoặc digest SHA-256 không khớp với bản ghi phát hành.

! Đây là rào chắn chính sách ở tầng CI/CD delivery hoặc Kubernetes admission webhook, không phải lỗi từ runtime của ứng dụng.

→ Bảo đảm image được build và ký số thông qua pipeline CI chính thức có attestation/provenance hợp lệ trước khi promote. [Ch18]

J08 Operation cannot be fulfilled: the object has been modified

- Operation cannot be fulfilled on <resource>: the object has been modified; please apply your changes to the latest version and try again

API server trả HTTP 409 Conflict khi update gửi metadata.resourceVersion không còn khớp version hiện hành. Đối chiếu API semantics; không cần giả định client thấy hay thao tác trực tiếp với bản ghi etcd.

! Đây là cơ chế kiểm soát đồng thời lạc quan (Optimistic Concurrency Control) của Kubernetes nhằm ngăn chặn ghi đè mất dữ liệu giữa các client cạnh tranh.

→ Sử dụng `k8s.io/client-go/util/retry.RetryOnConflict` để đọc lại snapshot mới nhất từ API Server và áp dụng thay đổi trước khi thử lại. [Ch22]

J09 no kind is registered for the type in scheme

- no kind is registered for the type <Type> in scheme <Scheme>

Lỗi cấu hình runtime (Kubernetes Scheme Error) khi một Go struct được truyền vào Controller Client hoặc Reconciler nhưng kiểu dữ liệu này chưa được đăng ký vào bảng `runtime.Scheme` thông qua hàm `AddToScheme`.

! Xảy ra phổ biến khi khởi tạo Operator với Custom Resource Definition (CRD) nhưng quên nạp `v1alpha1.AddToScheme(mgr.GetScheme())`.

→ Đăng ký `SchemeBuilder` của CRD vào `Manager Scheme` trước khi khởi động Controller. [Ch23]

J10 Denied by Supply Chain Policy: reachable vulnerability or untrusted builder

- reachable <SEVERITY> vulnerability <ID> in <package> (symbol: <symbol>)
- untrusted builder identity: <builder>

Trạng thái chặn theo policy khi công cụ phân tích tìm đường gọi có thể tới ký hiệu liên quan lỗ hổng, hoặc identity của builder không được chấp nhận.

! Reachability tĩnh không chứng minh đường ấy đã chạy, khai thác thành công hay có mã độc. Phân biệt kết quả phân tích với trace thực nghiệm.

→ Cập nhật phiên bản dependency đã vá lỗi, loại bỏ việc gọi ký hiệu chứa lỗ hổng, hoặc kiểm tra lại builder ID trong CI workflow. [Ch26]

J11 MCP Tool Execution Denied: unauthorized mutation or SSRF boundary violation

- mcp: tool authorization denied for role <role> on tool <tool>
- mcp: ssrf blocked: target ip <ip> belongs to private/metadata range

Trạng thái từ chối (MCP Security Boundary Status) khi Agent AI cố gắng kích hoạt công cụ làm thay đổi trạng thái hạ tầng (Mutating Tool) mà không có quyền hạn, hoặc truyền tham số URL vi phạm ranh giới phòng vệ SSRF (hướng tới AWS Metadata Service `169.254.169.254` hoặc Loopback `127.0.0.1`).

! Schema kiểm tra cấu trúc và constraint đã khai báo, không thay authorization, kiểm soát destination hay phòng prompt injection. Handler, gateway hoặc lớp mạng phải thực thi policy phù hợp; không phải yêu cầu bắt buộc dùng Go.

→ Kiểm tra role của Agent trong phiên MCP, cấu hình whitelist URL cho công cụ mạng, hoặc cấp token ủy quyền (Change Request ID) trước khi gọi các tool gây đột biến. [Ch28]

Tài liệu tham khảo: giữ điểm neo

Không thêm tài liệu vào bài học. Trích nguyên văn block @references hiện hành; link và version thuộc nội dung, không thuộc artwork.

Tài liệu tham khảo

1. Go Team. Frequently Asked Questions: Origins and Design.
go.dev/doc/faq
2. Cox, Russ. Toward Go 2. 13 July 2017. go.dev/blog/toward-go2
3. Go Team. Go 1 and the Future of Go Programs. March 2012.
go.dev/doc/go1compat
4. Gerrand, Andrew. Go 1.5 is released. 19 August 2015. go.dev/blog/go1.5
5. Bonventre, Andrew. Go 1.11 is released. 24 August 2018.
go.dev/blog/go1.11
6. Go Team. Go 1.18 is released! 15 March 2022. go.dev/blog/go1.18
7. Go Team. Go 1.27 Release Notes. 2026. go.dev/doc/go1.27

Code-copy laboratory

QA fixture, không phải nội dung bài học. Cùng payload được lưu trong copy_fixture.txt; có TAB, dấu tiếng Việt, hai dấu cách cuối dòng và dòng rỗng.

A. Text thông thường; TAB được trình bày bằng tab-stop bốn cột.

```
func main() {  
    fmt.Println("Tiếng Việt: ă â ê ô ơ ư đ")  
  
}
```

B. Cùng hình thức, thêm ActualText ở marked-content span.

```
func main() {  
    fmt.Println("Tiếng Việt: ă â ê ô ơ ư đ")  
  
}
```

Không có gutter, số dòng hoặc số dòng ẩn. Hiển thị đúng indentation không chứng minh clipboard giữ đúng TAB. Kết quả extract và giới hạn PDF reader ghi riêng trong copy_code_report.md.

Công phê duyệt

Catalog là bằng chứng prototype, không phải chứng nhận cả sách đã sẵn sàng in.

Duyệt hướng bìa A/B/C/D; duyệt hệ thống H1–H4 và Atlas 10.5 pt; duyệt vector hóa có kiểm chứng từng sơ đồ. Giữ hình phức tạp đọc được trước khi tối ưu số trang.

Hai điều kiện chưa đóng: clipboard chính xác trên hai PDF reader và thông số giấy/đóng gáy. Mẫu thiếu thử nghiệm không được gọi PASS. Sau phê duyệt phải có nhiệm vụ tích hợp riêng, candidate khác đường dẫn và regression toàn sách.